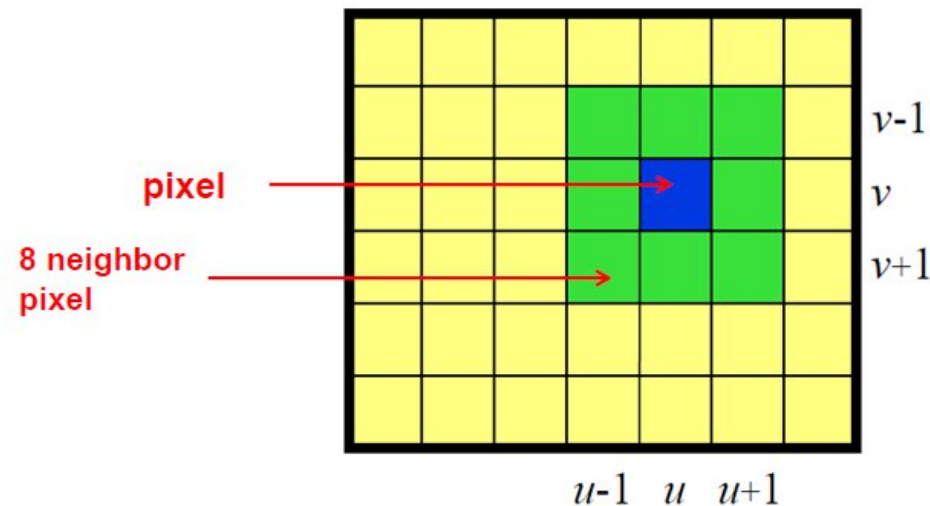
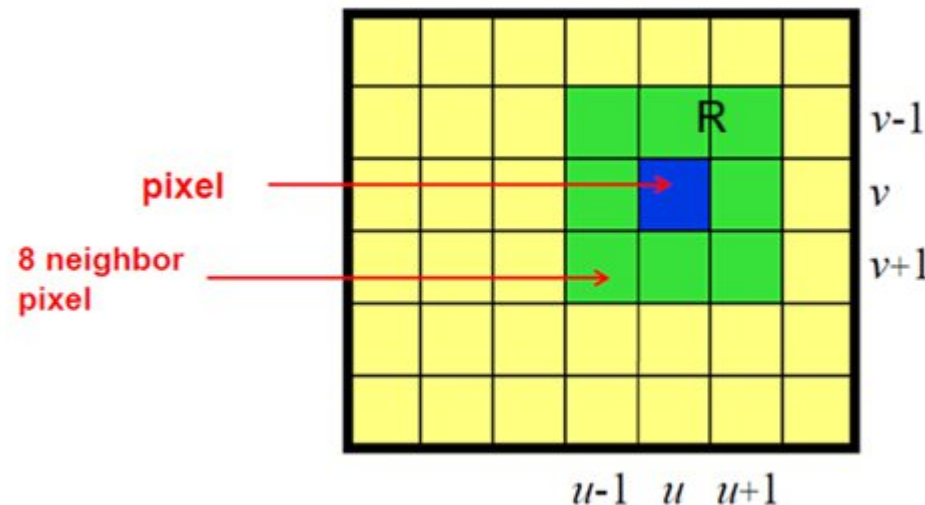


Filtri

- In questa lezione ci concentreremo sulle immagini in scala di grigi, anche se la maggior parte dei concetti possono essere estesi a immagini a colori
- Dato che gli operatori puntuali visti nella lezione precedente dipendono da un singolo pixel dell'immagine, l'informazione che prendono in input è limitata ($I(u,v)$) (e, quindi, limitate sono le cose che possono fare)
- Un filtro spaziale utilizza (anche) il valore dei pixel circostanti, selezionati tramite una finestra dell'immagine centrata sul pixel corrente

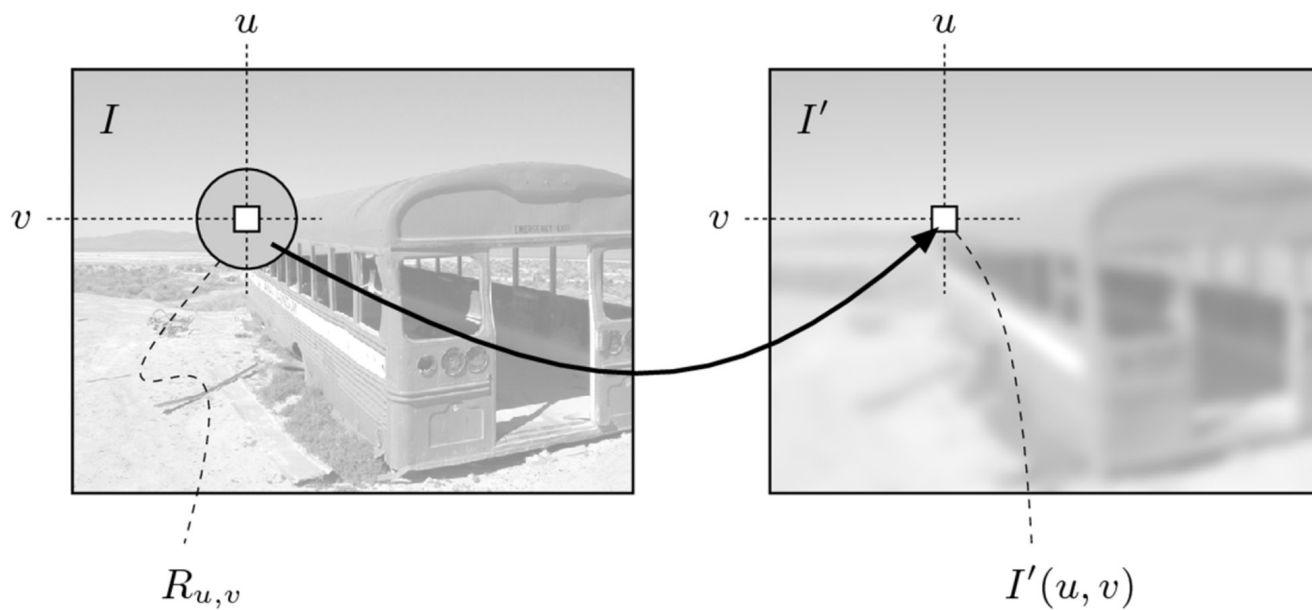


- Un filtro è un'operazione che combina il valore del pixel di un'immagine ($I(u,v)$) col valore dei pixel circostanti contenuti in una finestra locale R centrata su (u,v) ($R_{u,v} = \{I(u-1,v-1), I(u-1,v), \dots, I(u,v), \dots\}$)
- R può, in genere, avere una forma qualsiasi (non necessariamente rettangolare), ma è fissata per ogni filtro (cioè non cambia durante la sua applicazione)
- Posso esprimere l'applicazione di un filtro alle coordinate (u,v) come: $I'(u,v) = f(R_{u,v})$
- Il filtro viene fatto "scorrere" su tutta l'immagine e, per ogni posizione (u,v) , calcola $I'(u,v) = f(R_{u,v})$
- La differenza con un operatore puntuale è che, nel caso di quest'ultimo, l'operazione è $= f(I(u,v))$



Filtri: esempio

- Esempio: la media (eventualmente pesata) dei pixel in una finestra R centrata su (u,v) . In quest'esempio R ha una forma sferica (caso raro)



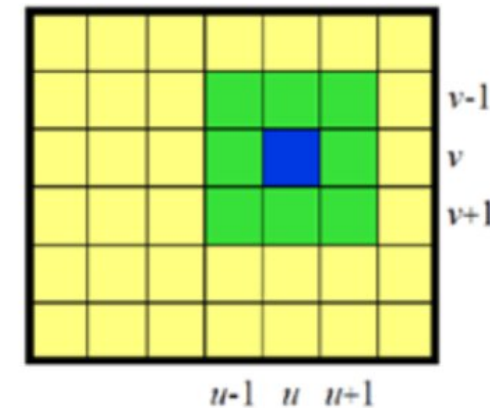
Esempio: Blurring con un filtro 3X3

- In quest'altro esempio il filtro ha una finestra di forma quadrata R di dimensione 3×3 e, come nel caso precedente, calcola la media di R
- Questo filtro, molto comune, è detto filtro di "Blurring" o di "Smoothing" o "Mean filter" o "Average filter"

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^1 \sum_{i=-1}^1 I(u+i, v+j)$$

$$\uparrow \\ f(R_{u,v})$$



- L'algoritmo itererà il calcolo di $I'(u, v)$ per ogni coppia di coordinate (u, v)

- Esempio: blurring



- Esempio: sharpening
- Sto usando la stessa finestra R di prima ma ho cambiato la funzione $f()$, che ora non calcola più la media ma evidenzia i dettagli dell'immagine



- Esempio: creare dei “pattern” artistici combinando localmente il valore dei pixel

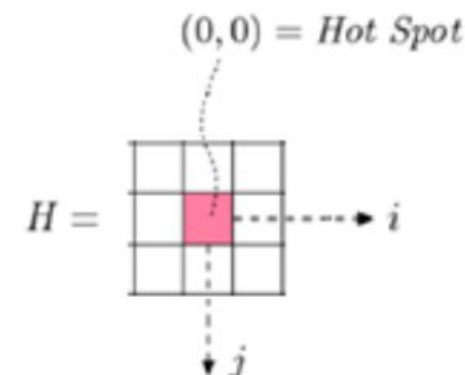
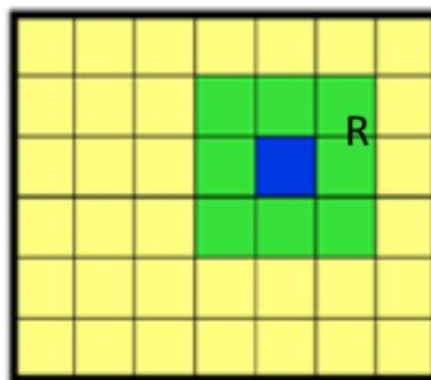


- Filter shape: non necessariamente una finestra quadrata (può essere rettangolare, sferica, ...)
- Filter size (nel caso di shape rettangolare): 3x3, 3x5, 5x5, 7x5, 7x7, ..., 21x21, ...
- Filter function $f()$: può essere **lineare** (somma pesata dei valori in R) o non lineare (funzione qualsiasi dei valori in R)
- In questa lezione approfondiremo i filtri lineari (che sono i più comuni), mentre nella prossima vedremo degli esempi di filtri non lineari

- Un filtro lineare si basa su una *filter function* $f()$ lineare, ovvero:
- $f(R_{u,v}) = w_1 I(u-1,v-1) + w_2 I(u-1,v) + \dots$
- Dove $w_1, w_2, \dots$ sono i *pesi* del filtro

- Il “Kernel” (detto anche “Filter Matrix” o “Filter Mask”) H contiene il valore dei *pesi* che caratterizzano lo specifico filtro lineare
- Il kernel è una matrice di valori reali che ha la stessa dimensione e forma di R (ma non è R !)
- L’esempio qui sotto mostra il kernel del filtro di blurring 3x3 dell’esempio precedente
- Per comodità notazionale l’origine del sistema di coordinate di H (detta “hot spot”) è posta al centro della matrice (tra poco si capirà perchè)

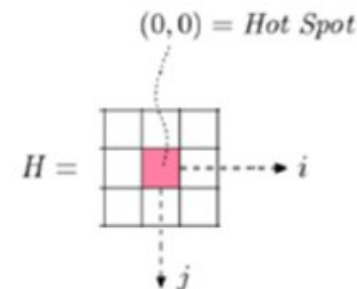
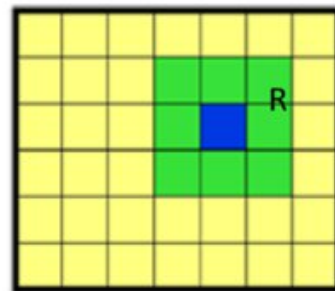
$$H(i,j) = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



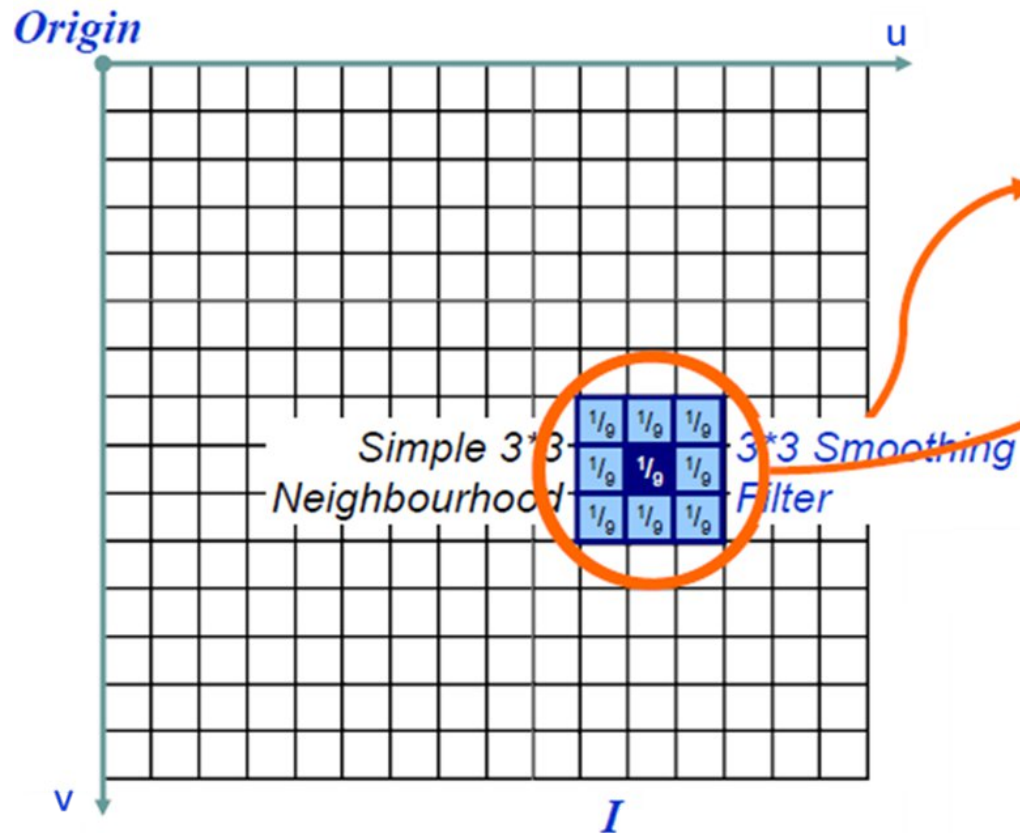
- Usando il kernel posso esprimere la filter function come una somma di prodotti tra i singoli valori di R e i corrispondenti valori di H
- Ad esempio, se H ed R sono entrambe matrici quadrate di dimensione $2k + 1 \times 2k + 1$, allora:

$$f(R_{u,v}) = \sum_{i=-k}^k \sum_{j=-k}^k I(u+i, v+j)H(i, j)$$

- Nella sommatoria di sopra la coppia (i,j) indicizza un elemento (peso) specifico di H usando l'hot spot come origine del sistema di riferimento in H e, contemporaneamente, è un "offset" rispetto alle coordinate (u,v) nell'immagine I



Esempio: Blurring con un filtro 3X3



104	100	108
99	106	98
95	90	85

R

*

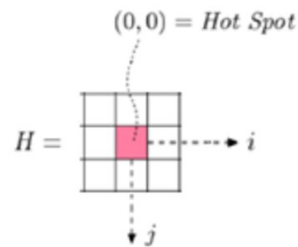
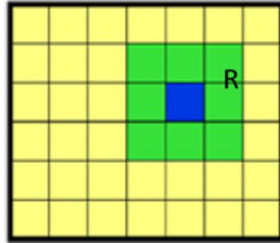
1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

H

$$\begin{aligned} f(R_{u,v}) &= \frac{1}{9} * 104 + \frac{1}{9} * 100 + \frac{1}{9} * 108 + \\ &\quad \frac{1}{9} * 99 + \frac{1}{9} * 106 + \frac{1}{9} * 98 + \\ &\quad \frac{1}{9} * 95 + \frac{1}{9} * 90 + \frac{1}{9} * 85 \\ &= 98.3333 \end{aligned}$$

Confronto tra i due metodi per calcolare la filter function (con e senza kernel H)

$$H(i, j) = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



$$f(R_{u,v}) = \sum_{i=-k}^k \sum_{j=-k}^k I(u+i, v+j) H(i, j)$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^1 \sum_{i=-1}^1 I(u+i, v+j)$$

$\uparrow$
 $f(R_{u,v})$

Il vantaggio di esprimere un filtro lineare tramite il suo kernel (a sinistra nell'immagine qui sopra) è che posso usare un **unico** algoritmo (detto **Convoluzione**) per applicare qualsiasi filtro lineare ad un'immagine e cambiare filtro semplicemente cambiando il kernel H

L'applicazione di un filtro lineare ad un'immagine è detta Convoluzione:

1. For $v = 1, \dots, N$:

2. For $u = 1, \dots, M$:

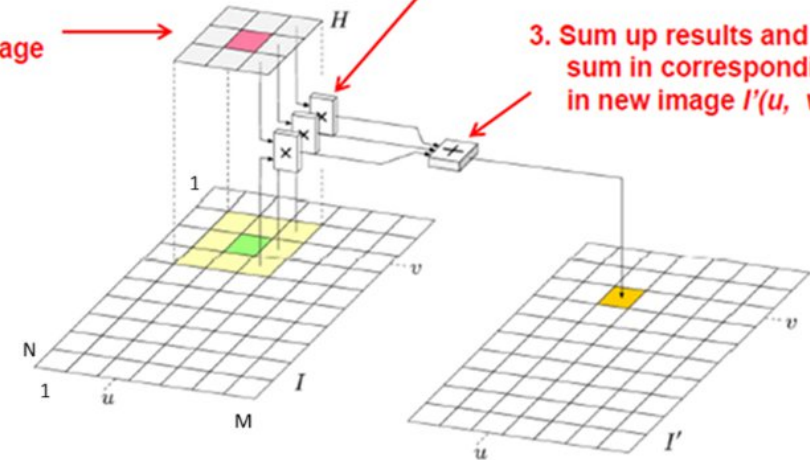
3.
$$I'(u,v) = \sum_{i=-k}^k \sum_{j=-k}^k I(u+i, v+j)H(i,j)$$

For each image position $I(u,v)$:

1. Move filter matrix H over image such that $H(0,0)$ coincides with current image position (u,v)

2. Multiply all filter coefficients $H(i,j)$ with corresponding pixel $I(u+i, v+j)$

3. Sum up results and store sum in corresponding position in new image $I'(u, v)$



Esempio

$$2*1+4*3+6*2+24*2+9*5+13*3+1*7+6*1+8*6 = 219$$

3	5	9	1	10
13	2	4	6	11
16	24	9	13	1
7	1	6	8	3
8	4	9	1	9

Immagine

X

1	3	2
2	5	3
7	1	6

H

=

		219		

Risultato

Esempio

$$4 * 1 + 6 * 3 + 11 * 2 + 9 * 2 + 13 * 5 + 1 * 3 + 6 * 7 + 8 * 1 + 3 * 6 = 198$$

3	5	9	1	10
13	2	4	6	11
16	24	9	13	1
7	1	6	8	3
8	4	9	1	9

Immagine

X

1	3	2
2	5	3
7	1	6

H

=

		219	198	

Risultato



0	0	0
0	1	0
0	0	0

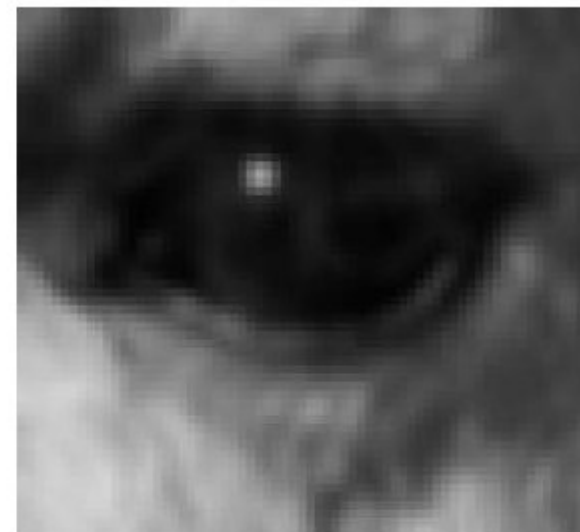


Identity function

Esempio


$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



Mean

Esempio: effetto della filter size su un mean filter



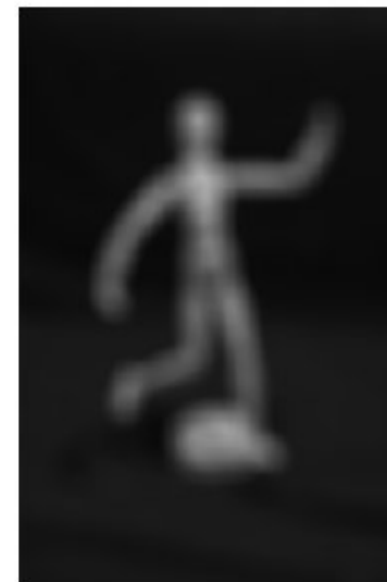
Original



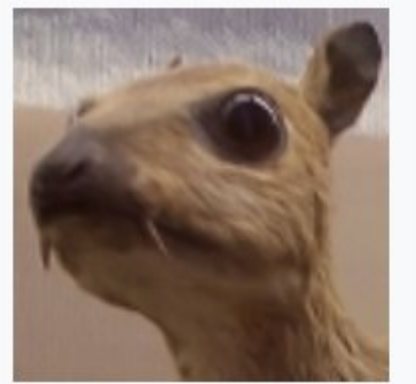
7×7



15×15



41×41



$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

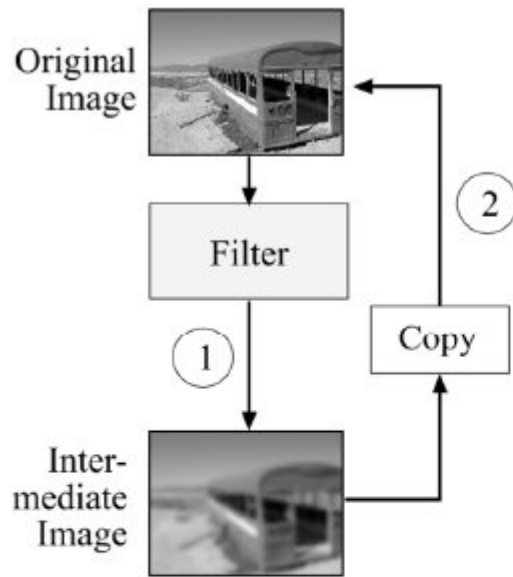


Sharpening

- Quest'esempio mostra che i pesi in H possono assumere valori qualsiasi, inclusi valori negativi
- Inoltre, anche se spesso i valori dei pesi hanno una certa simmetria rispetto all'hot spot, **non** è questa una condizione necessaria per avere un filtro lineare

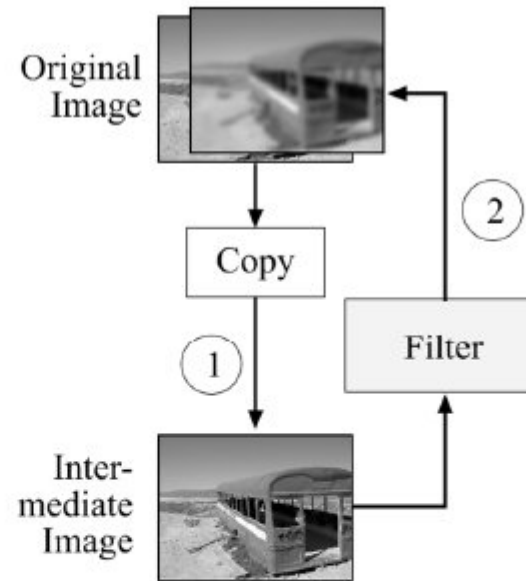
Applicare un filtro lineare ad un'immagine attraverso la Convoluzione lineare

Da un punto di vista implementativo, mentre il filtro convolutivo viene “fatto scorrere” sull'immagine, non posso sovrascrivere il valore dei vecchi pixel



Version A

Store results I' in intermediate image, then copy back to replace I

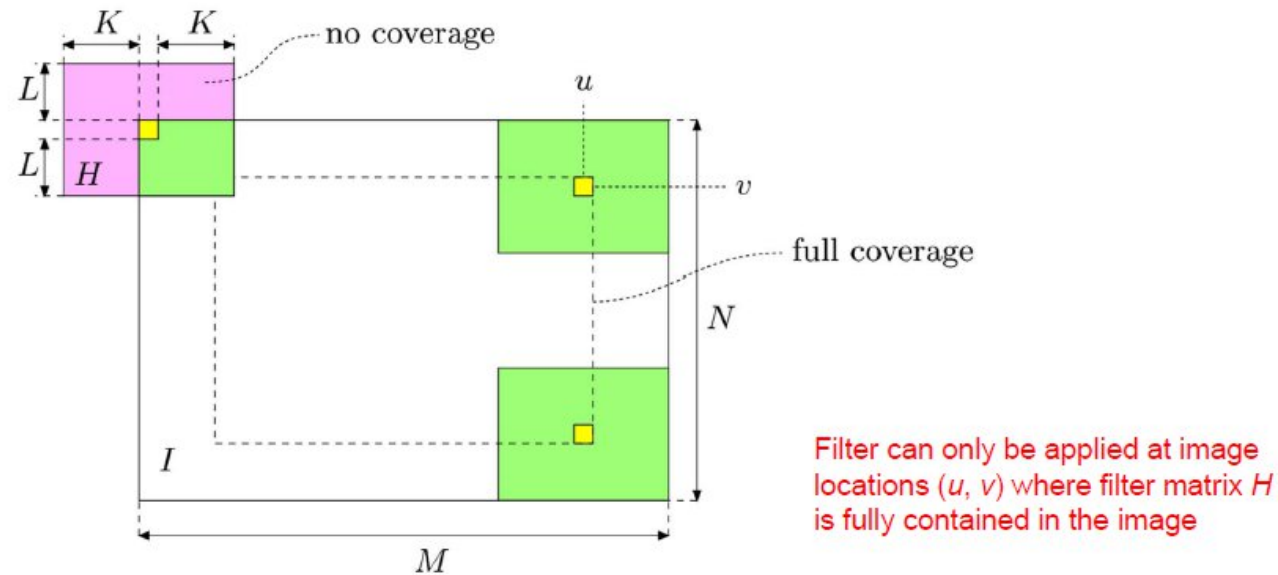


Version B

Copy original image I to intermediate image, use it as source, then store results I' to replace original image

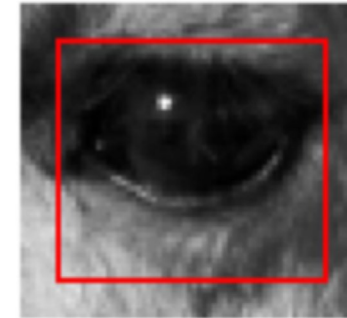
For a filter of size $(2K+1) \times (2L+1)$, if image size is $M \times N$, filter is computed over the range:

$$K \leq u' \leq (M-K-1) \quad \text{and} \quad L \leq v' \leq (N-L-1)$$

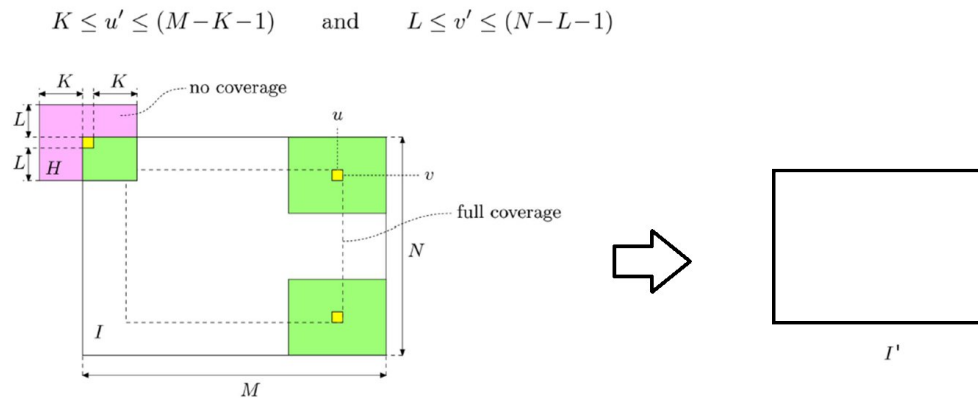


Trattamento dei bordi

- Se il filtro non può essere applicato ai bordi, cosa fare?
- Possibili soluzioni:
 - Cropping: *ritaglio* un'immagine più piccola escludendo i bordi di larghezza K e altezza L . Restituisce un'immagine I' più piccola



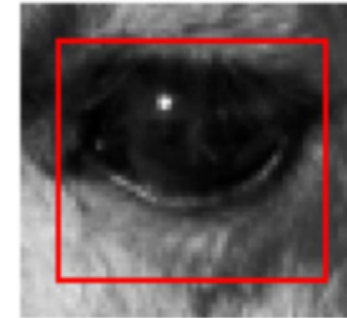
Crop



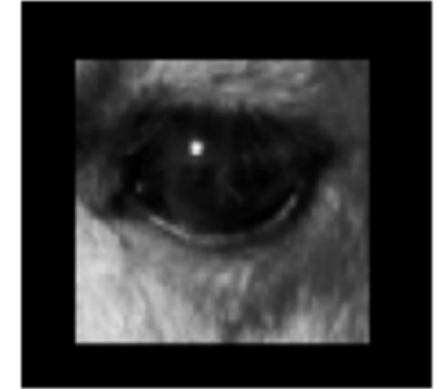
Cropping

Trattamento dei bordi

- Se il filtro non può essere applicato ai bordi, cosa fare?
- Possibili soluzioni:
 - Cropping: *ritaglio* un'immagine più piccola escludendo i bordi di larghezza K e altezza L . Restituisce un'immagine I' più piccola
 - Padding: *aggiungo* una cornice all'immagine di larghezza K e altezza L prima di applicare il filtro
 - Zero padding (aggiungo una cornice di pixel tutti con valore 0)
 - Mirror/Simmetric padding (aggiungo una cornice effettuando il mirroring dell'immagine su ogni lato)
 - Replicate padding (aggiungo una cornice replicando il valore del pixel "reale" più vicino)



Crop



Zero Padding

- Lo schema visto finora è abbastanza generale e flessibile: per implementare filtri diversi posso applicare lo stesso algoritmo di convoluzione cambiando solo H
- I filtri lineari, quindi, differiscono per il kernel H (e per la finestra R)
- Kernel diversi servono a produrre trasformazioni diverse dell'immagine
- Vedremo ora alcuni esempi e altri li studieremo più avanti

Weighted Smoothing Filters

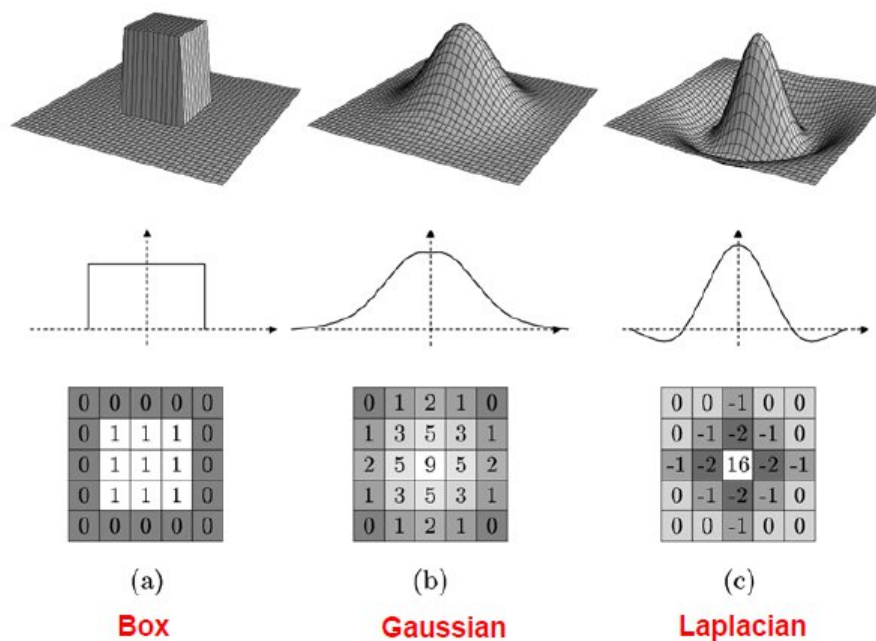
- Lo smoothing è più efficace se nel kernel uso pesi con valore non uniforme, dando più importanza alla posizione centrale (hot spot)
- Questo smoothing con pesi non uniformi è detto anche “weighted averaging”

$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$
$\frac{2}{16}$	$\frac{4}{16}$	$\frac{2}{16}$
$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$

Weighted
averaging filter

Rappresentazione 3D di un kernel

- L'immagine qui sotto mostra filtri diversi. Per essere più precisi, mostra i loro kernel utilizzando sia una rappresentazione in forma di matrici H , sia una rappresentazione 3D (valori più "alti" corrispondono a pesi maggiori)
- Ma come fare per calcolare i pesi di un kernel? Ad esempio, come sono stati calcolati i pesi del kernel Gaussiano?

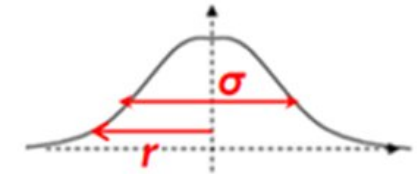
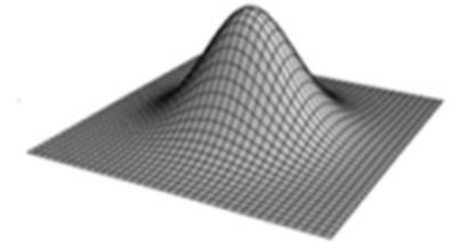
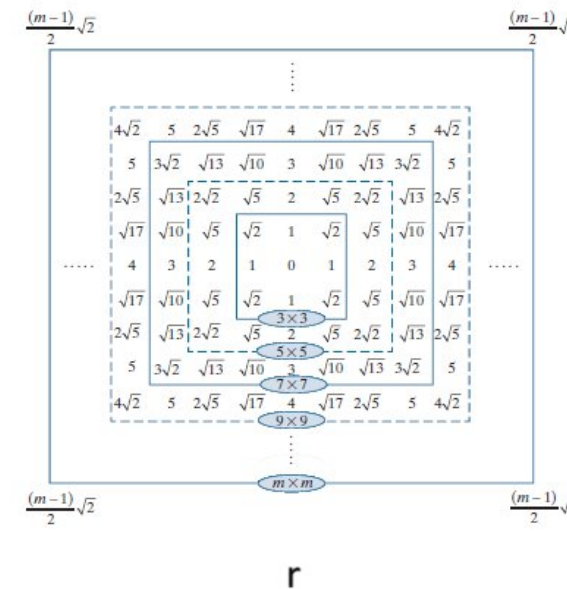


Filtro Gaussiano

- Posso esprimere il valore del generico peso (i,j) di H , kernel Gaussiano, usando la definizione di funzione Gaussiana (bidimensionale):

$$H(i,j) = Ke^{-\frac{i^2+j^2}{2\sigma^2}} = Ke^{-\frac{r^2}{2\sigma^2}}$$

- dove:
 - r è la distanza di (i,j) dall'hot spot
 - σ è la "larghezza" della curva a campana (formalmente, è la sua deviazione standard)
 - K è una costante che serve ad esprimere il valore del peso massimo (quello corrispondente all'hot spot). Nell'esempio accanto, $K=9$
- Dopo di che non devo fare altro che calcolare $H(i,j)$ usando la formula di sopra per ogni valore di (i,j) . Tipicamente, il valore di ogni peso viene poi approssimato all'intero più vicino

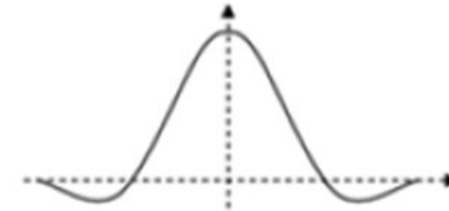
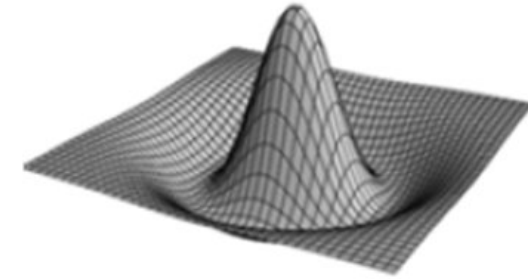


0	1	2	1	0
1	3	5	3	1
2	5	9	5	2
1	3	5	3	1
0	1	2	1	0

Gaussian
filter

Kernel calcolati a partire da forme analitiche

- Lo stesso procedimento può essere applicato per il calcolo di kernel diversi, purchè conosca una forma analitica che descrive il valore dei pesi
- In particolare, il kernel “Laplaciano” accanto ha anche dei valori negativi e può essere usato, ad esempio, per un filtro di sharpening (in alternativa a quello visto precedentemente)



0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

Laplacian
filter

Definizione formale della Convoluzione



Formalmente, la convoluzione è un'operazione tra due funzioni, $I()$ e $H()$ (dove non solo l'immagine I ma anche il kernel H sono interpretati come funzioni) che restituisce una terza funzione $I'()$ e si indica con l'operatore $*$

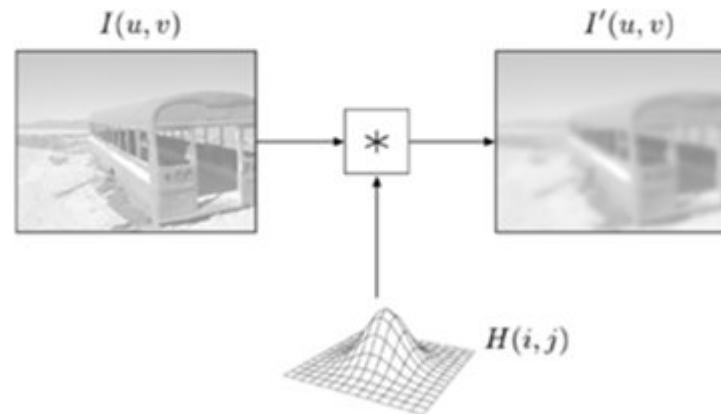
$$I'(u, v) = (I * H)(u, v) = \sum_{i=-k}^k \sum_{j=-k}^k I(u + i, v + j) H(i, j)$$

Definizione formale della Convoluzione

- A volte conviene usare la seguente definizione alternativa, dove sia la funzione $I()$ che la funzione $H()$ sono definite su un dominio (bidimensionale) infinito (il prodotto cartesiano degli interi: $\mathbb{Z}^2$) e centrate in $(0,0)$:

$$I'(u, v) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} I(u+i, v+j) \cdot H(i, j)$$

$$I' = I * H$$



- In pratica, è come se applicassimo uno “zero-padding” infinito sia ad H che ad I

- Usando la definizione precedente (dominio infinito), è possibile dimostrare le seguenti proprietà della convoluzione:

- Commutativity

$$I * H = H * I$$

Same result if we convolve
image with filter or vice versa

- Linearity

$$(s \cdot I) * H = I * (s \cdot H) = s \cdot (I * H)$$

If image multiplied by scalar
Result multiplied by same scalar

$$(I_1 + I_2) * H = (I_1 * H) + (I_2 * H)$$

If 2 images added and convolve
result with a kernel H ,
Same result if each image
is convolved individually + added

- Associativity

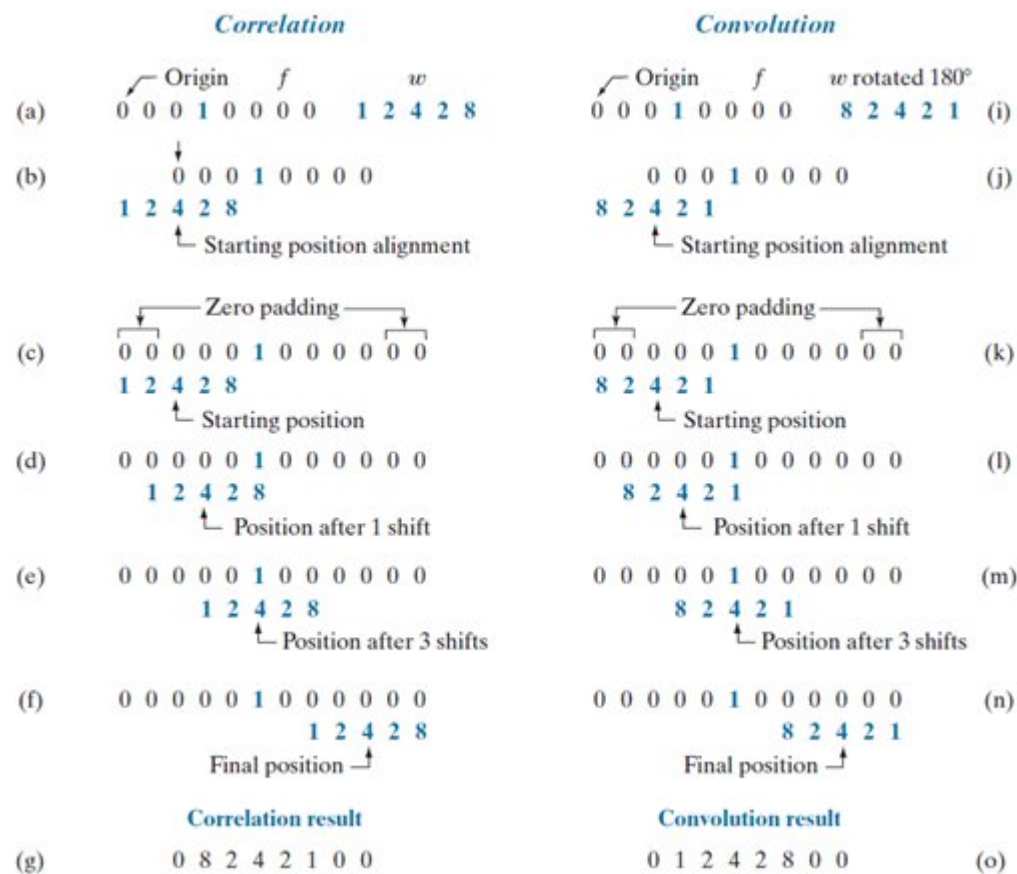
$$A * (B * C) = (A * B) * C$$

Order of filter application irrelevant
Any order, same result

Differenza tra Convoluzione e Correlazione

- "Convoluzione" e "Correlazione" (lineare) sono spesso usati come sinonimi, anche se, strettamente parlando, nella convoluzione il kernel viene ruotato prima di essere applicato. Ad esempio, in un segnale 1D, viene ruotato di 180 gradi.
- Il motivo di ciò è che, se il segnale è costituito da un unico impulso discreto, con la correlazione (cioè, senza rotazione iniziale del kernel), si ottiene come risultato il kernel ruotato di 180 gradi, mentre con la convoluzione si ottiene il kernel originario...

Differenza tra Convoluzione e Correlazione: esempio (1D)



Differenza tra Convoluzione e Correlazione

- Se il kernel è simmetrico rispetto alla posizione centrale, allora la Convoluzione e la Correlazione sono identiche
- Senza perdita di generalità, continueremo a parlare di "convoluzione", specificando se e quando il filtro è pre-ruotato

Convoluzione come prodotto scalare

- Applicare un filtro convolutivo *in una specifica posizione* (u,v) dell'immagine equivale a considerare R ed H come dei vettori $\mathbf{r}$ ed $\mathbf{h}$ che "linearizzano", rispettivamente, il contenuto di R ed H (v. figura sotto) e calcolarne il prodotto scalare
- La convoluzione quindi corrisponde a calcolare tale prodotto per ogni posizione (u,v)

